Chapter 7
Convolutional Neural Networks

Asst. Prof.Rapeeporn Chamchong, PhD
Department of Computer Science, Faculty of Informatics, MSU

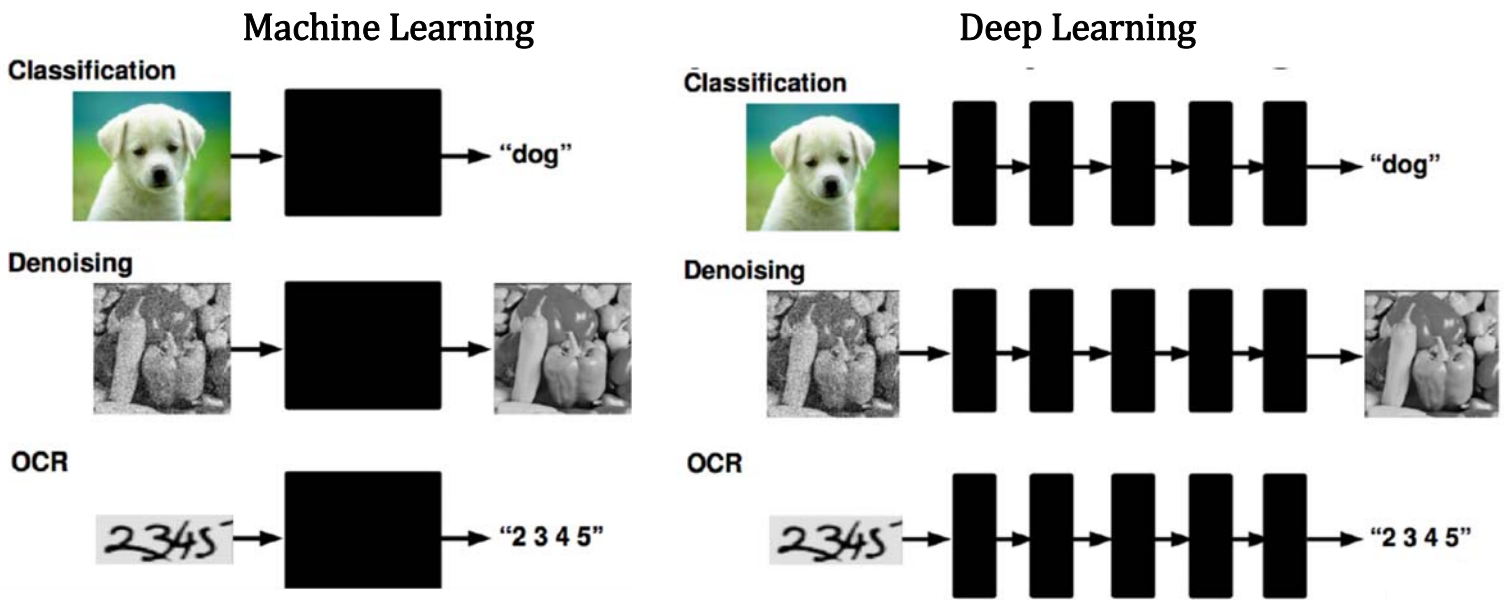
Outlines

- Recap: ML & DL
- Convolutional Neural Networks (CNNs)
- Structure of CNNs
- ImageNet
- Architectures of CNNs from ImageNet

What is Deep Learning?

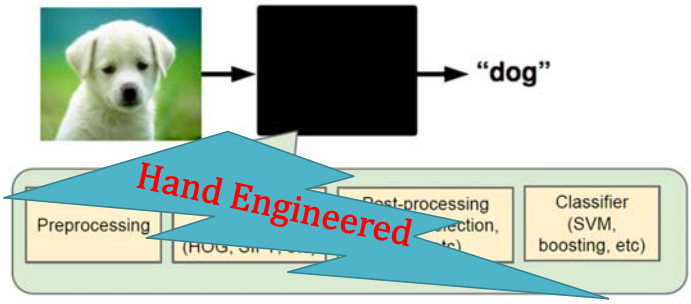
- Composition of non-linear transformation of the data.
- Goal: **Learn useful representations, aka features, directly from data.**
  - Many varieties can be unsupervised or supervised.
  - Today is about CNN, a **supervised** deep learning method.

Recap: Supervised Learning



Deep Learning

- So deep learning is about learning feature representation in a compositional manner.
- But wait, **why learn features**?



- Most critical for accuracy
- Most time-consuming in development
- What is the best feature???
- What is next?? Keep on crafting better features?

❖ Let's learn feature representation directly from data.

Learn features and classifier together

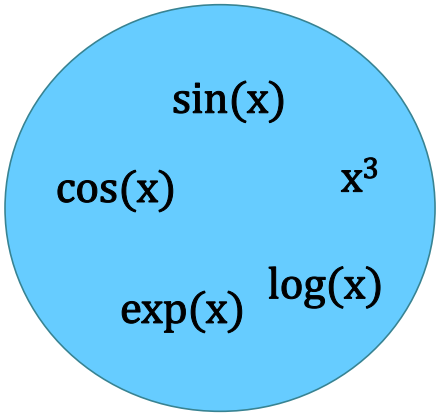
- Learn an end-to-end recognition system.
- A non-linear map that takes raw pixels directly to labels.

Q: How can we build such a highly non-linear system?

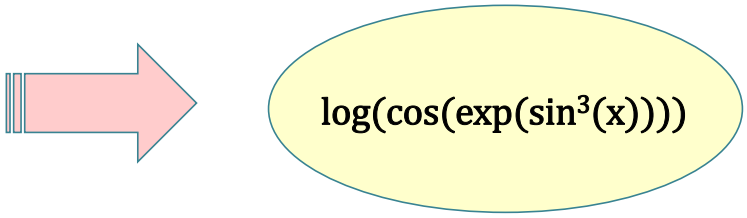
A: By combining simple building blocks, we can make more and more complex systems.

Building a complicated function

Simple functions



One example of complicated function

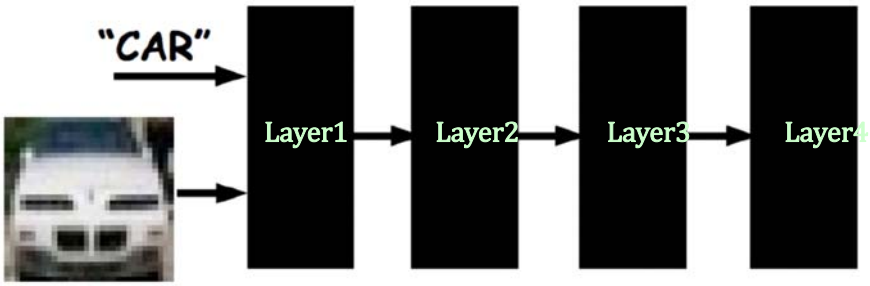


- Composition is at the core of deep learning methods
- Each "simple function" will have parameters subject to learning

Intuition behind Deep Neural Nets

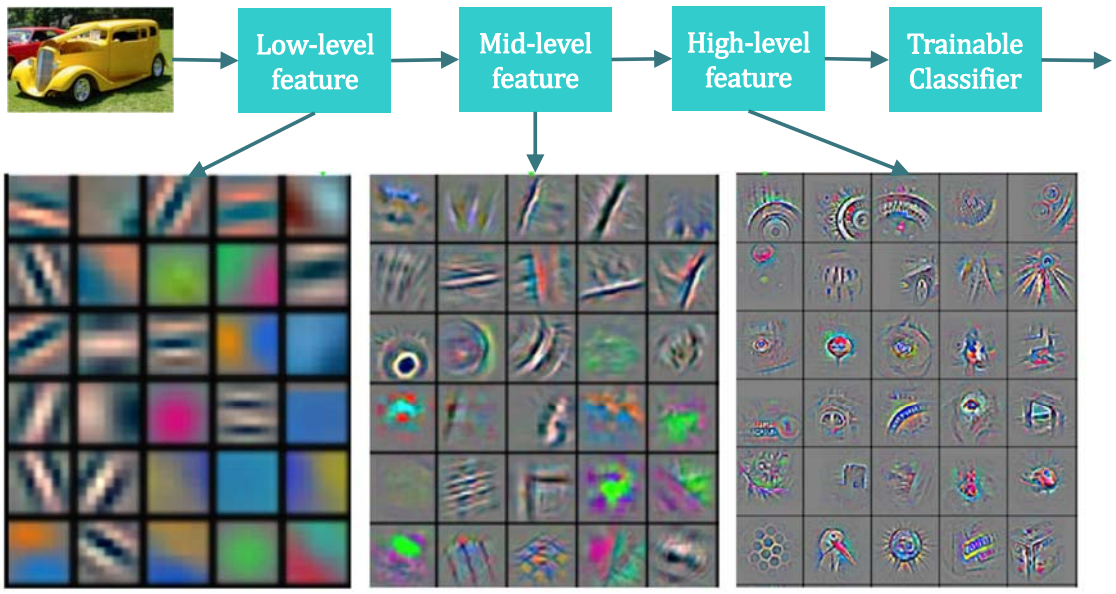
Note:

- Each black box can have trainable parameters.
- Their Composition makes a highly non-linear system.



- The final layer outputs a probability distribution of categories.

Deep Learning = Learning Hierarchical Representations



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

A simple single layer Neural Network

- Consists of a linear combination of input through a nonlinear function:

$$z = Wx + b$$
$$a = f(z)$$

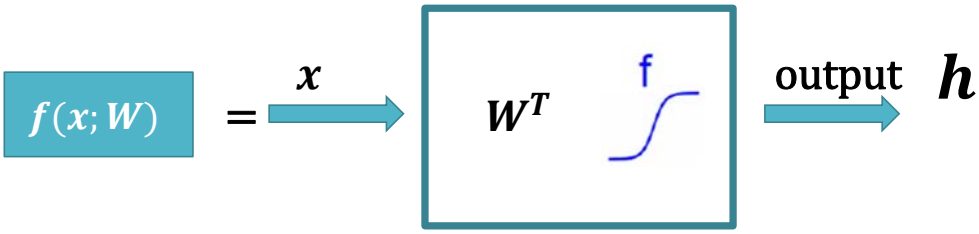
W is the weight parameter to be learned.

x is the output of the previous layer

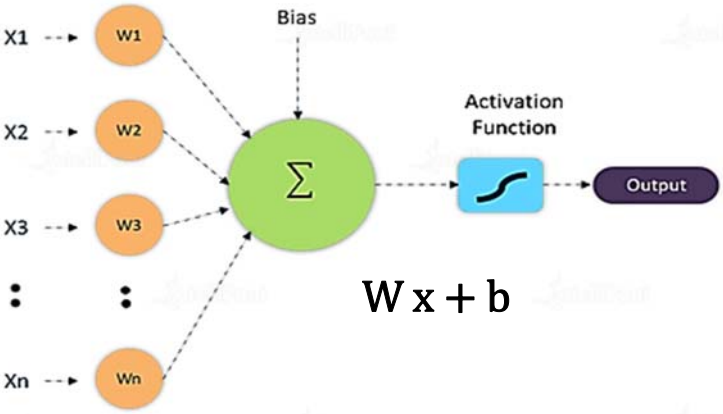
f is a simple nonlinear function.

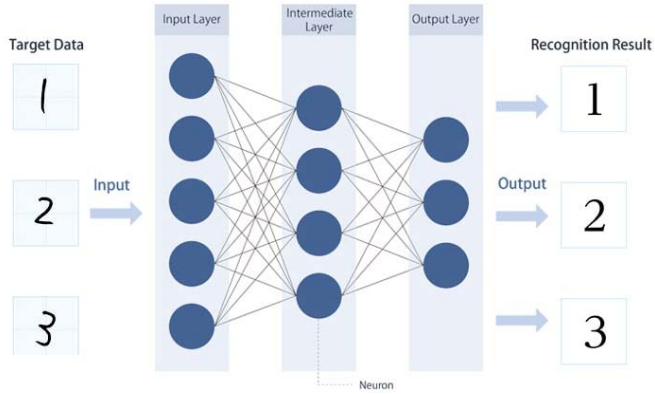
Popular choice of activation function (a) is max(x,0), called ReLU (Rectified Linear Unit)

Graphical Representation

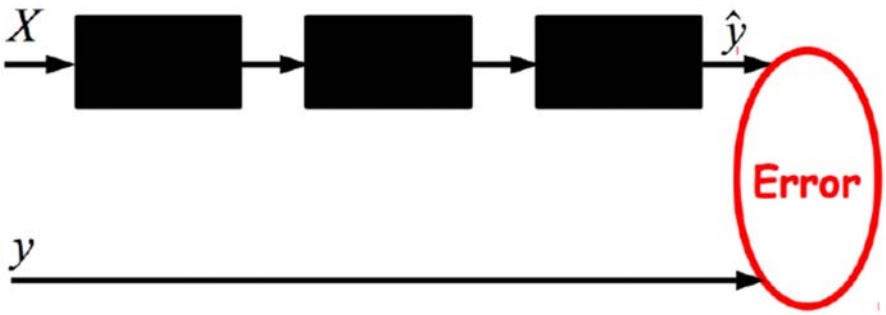


h is called a neuron, Hidden unit or feature.





Joint training architecture overview



Note: Multi-layer neural nets with more than two layers are nowadays called deep nets!!

Note: User must specify the number of layers, number of hidden units, type of layers and loss function.

Neural Net Training

- Compute loss on small mini-batch
- Compute gradient w.r.t. parameters
- Use gradient to update parameters

$$J(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \mathcal{L}\big(\underbrace{f(x^{(i)};\theta)}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}}\big)$$

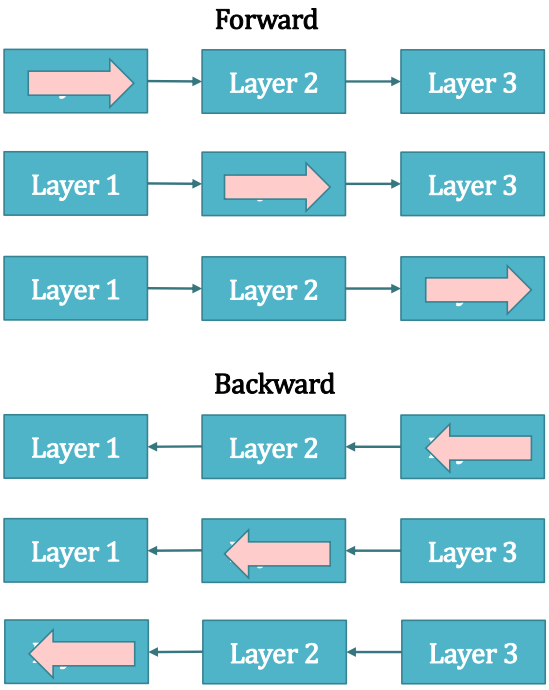
$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}}{\operatorname{argmin}}\, J(\boldsymbol{\theta})$$

Remember:

Optimization through gradient descent

$$\theta \leftarrow \theta - \eta \frac{\partial J(\theta)}{\partial \theta}$$

How can we set the learning rate?

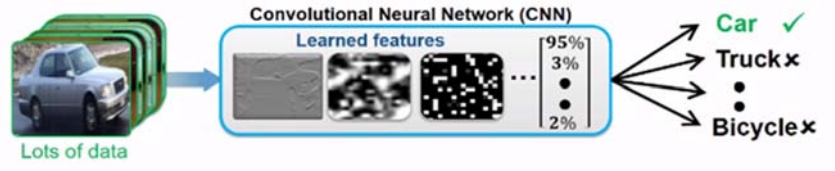


Types of DL

- Unsupervised Learning
  - Autoencoders
  - Deep Belief Net (DBN)
- Supervised Learning
  - Convolutional Neural Networks (CNN)
  - Recurrence Neural Networks (RNN)
  - Transformer
  - Reinforcement Learning to Neural Networks

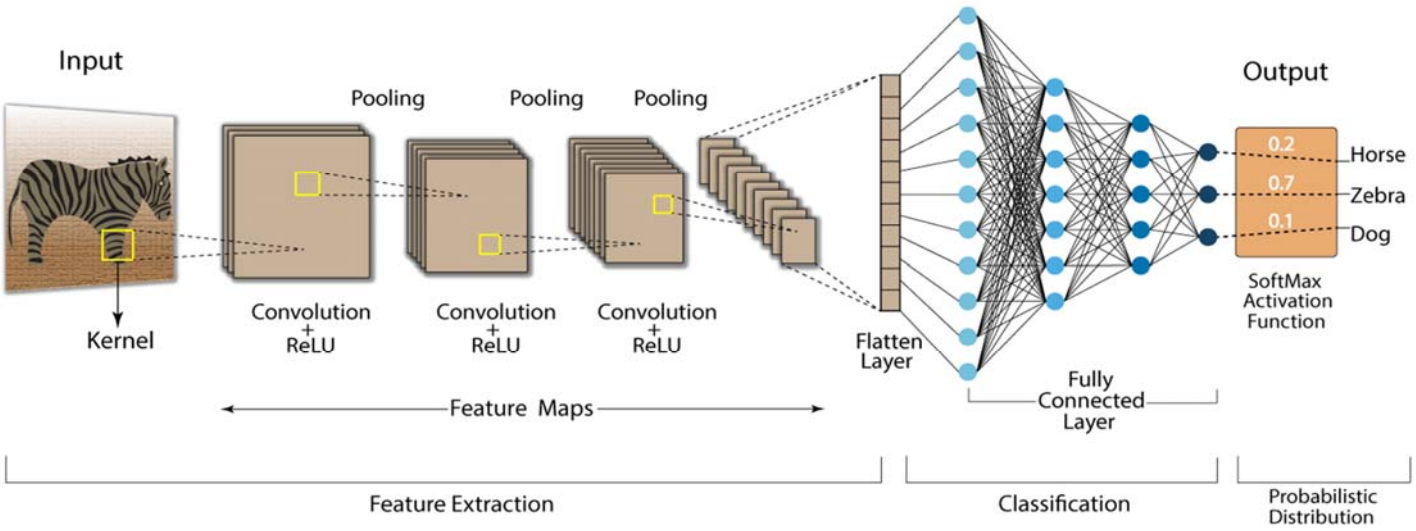
Approaches for DL

- Train a DL from scratch
  - Recommended only when:
    - *Data: 1000s to millions of labeled images*
    - *Computation: Require GPU*
    - *Training time: Days to weeks for real problems*
    - *Accuracy: High (can overfit to small datasets)*



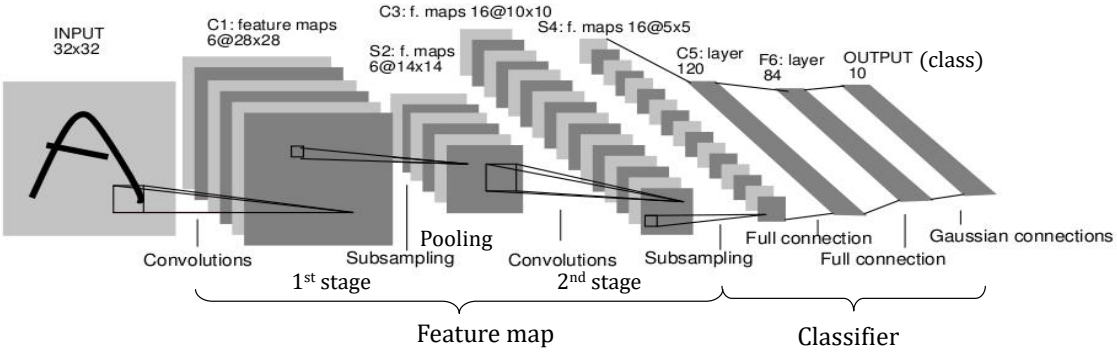
- Fine-tune a pre-trained model (transfer learning)

Convolutional Neural Network (CNN, ConvNets)

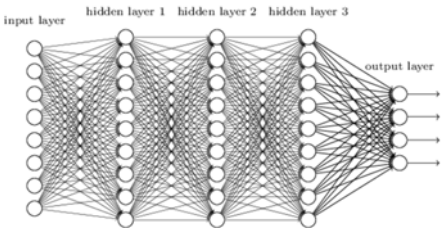


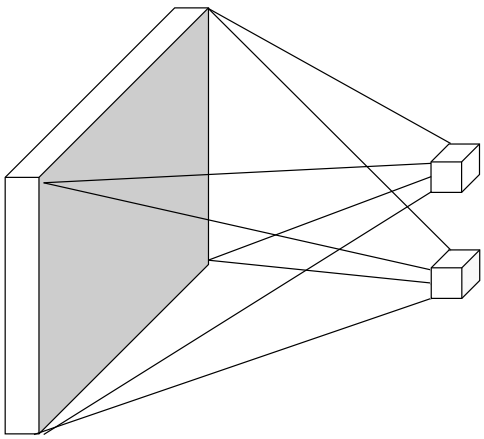
ConvNets

- LeNet-5 proposed by Lecun (1998).
- CNNs
  - Convolution layer
  - Pooling layer
  - Fully connected layer



Convolutional layer anatomy

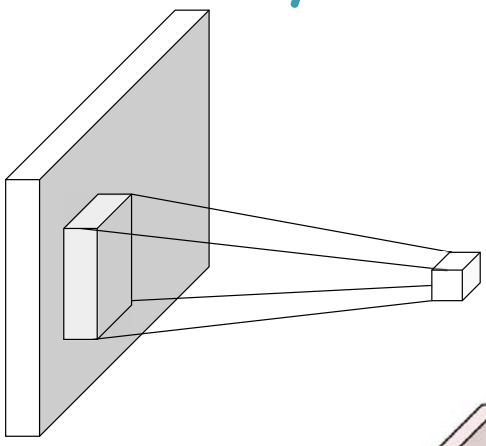




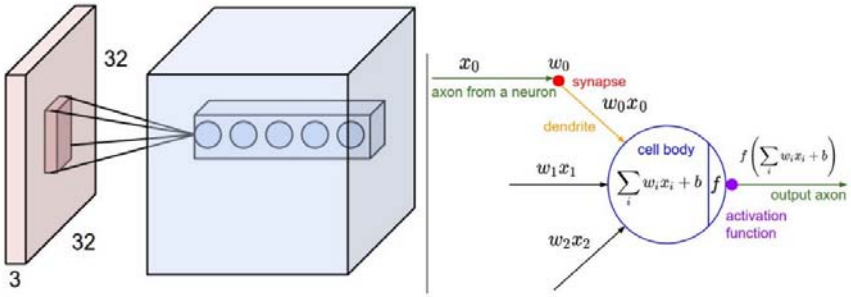
image

Fully connected layer

Convolutional layer anatomy



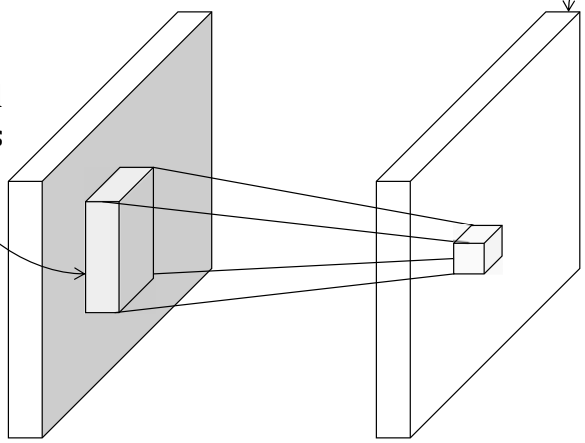
image



Convolutional layer anatomy

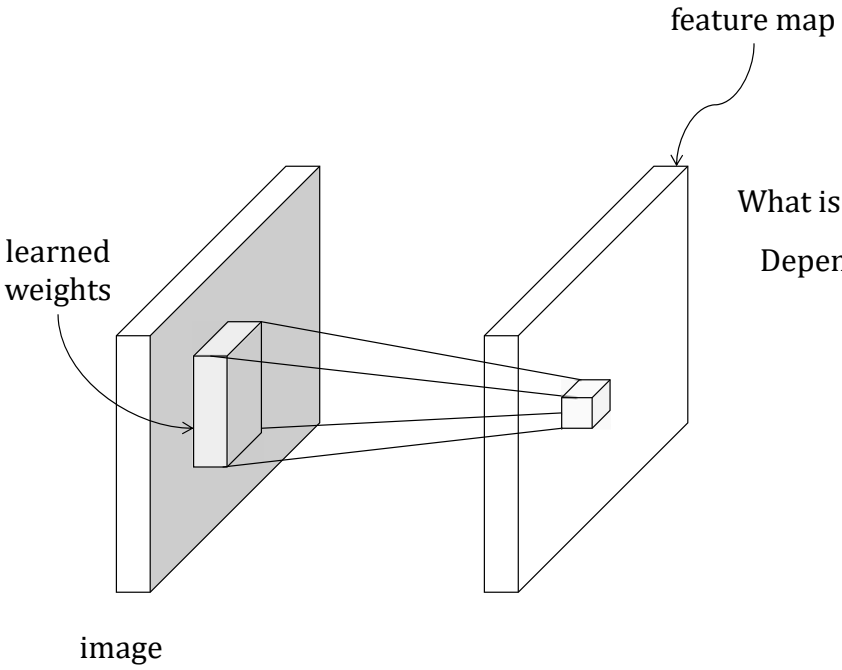
feature map

learned weights



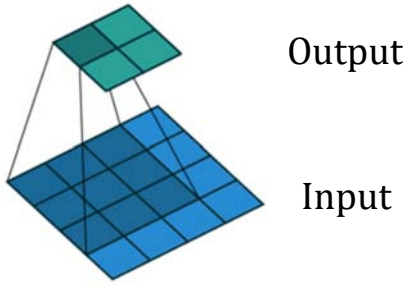
image

Convolutional layer anatomy



What is the feature map resolution?

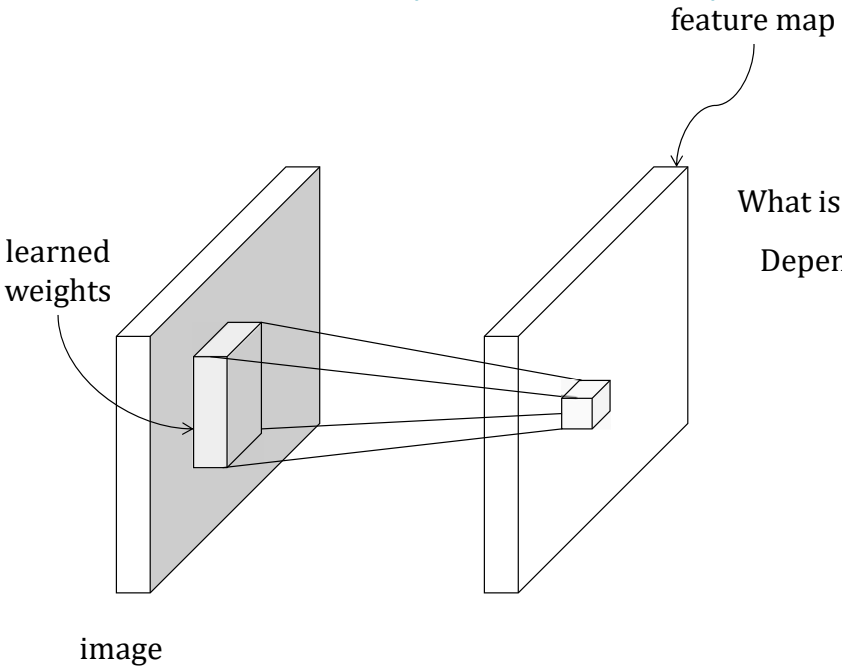
Depends on *padding* and *stride*:



No padding, stride 1

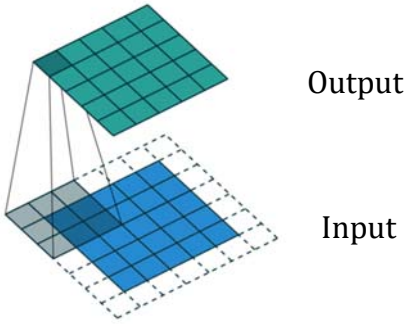
Animation source

Convolutional layer anatomy



What is the feature map resolution?

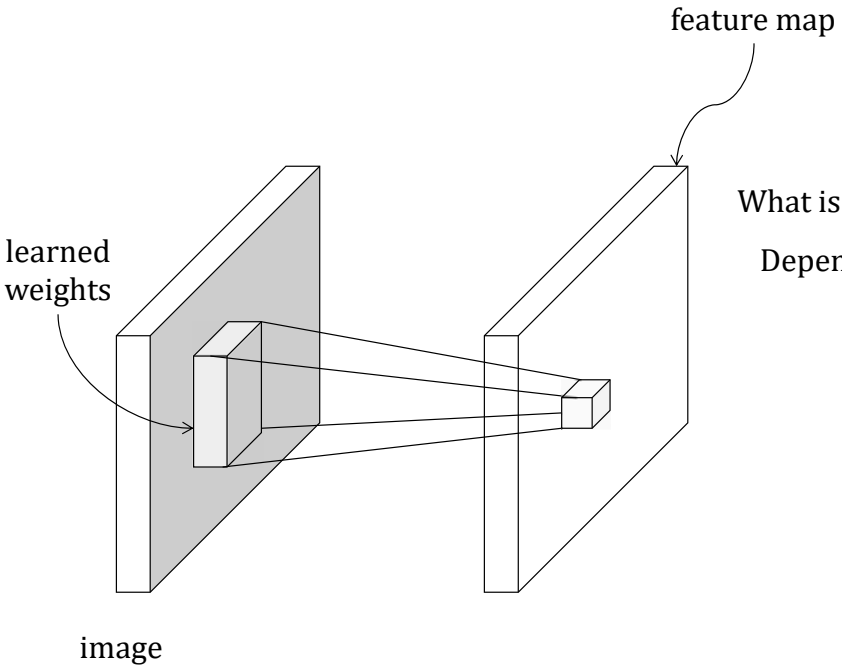
Depends on *padding* and *stride*:



With padding, stride 1

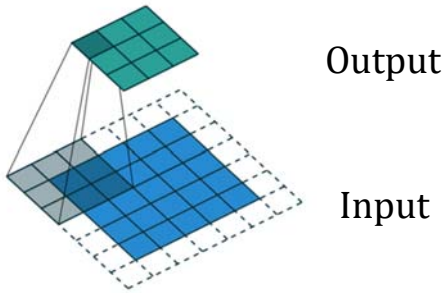
Animation source

Convolutional layer anatomy



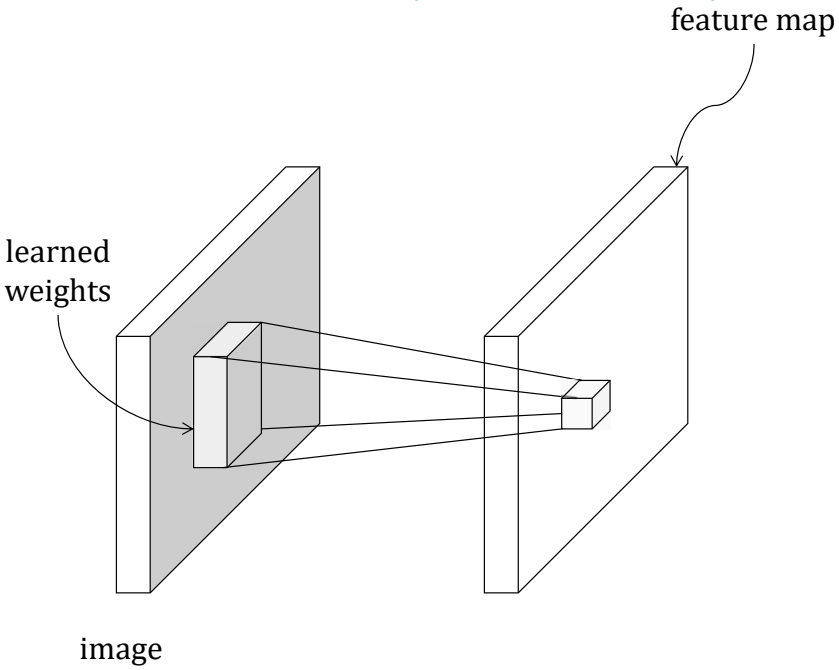
What is the feature map resolution?

Depends on *padding* and *stride*:

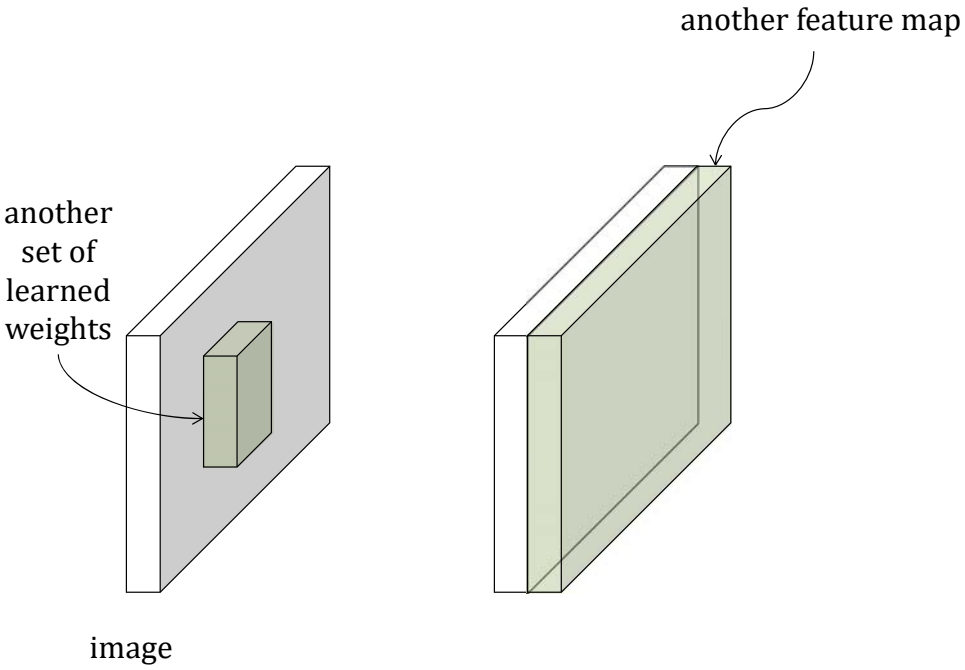


With padding, stride 2

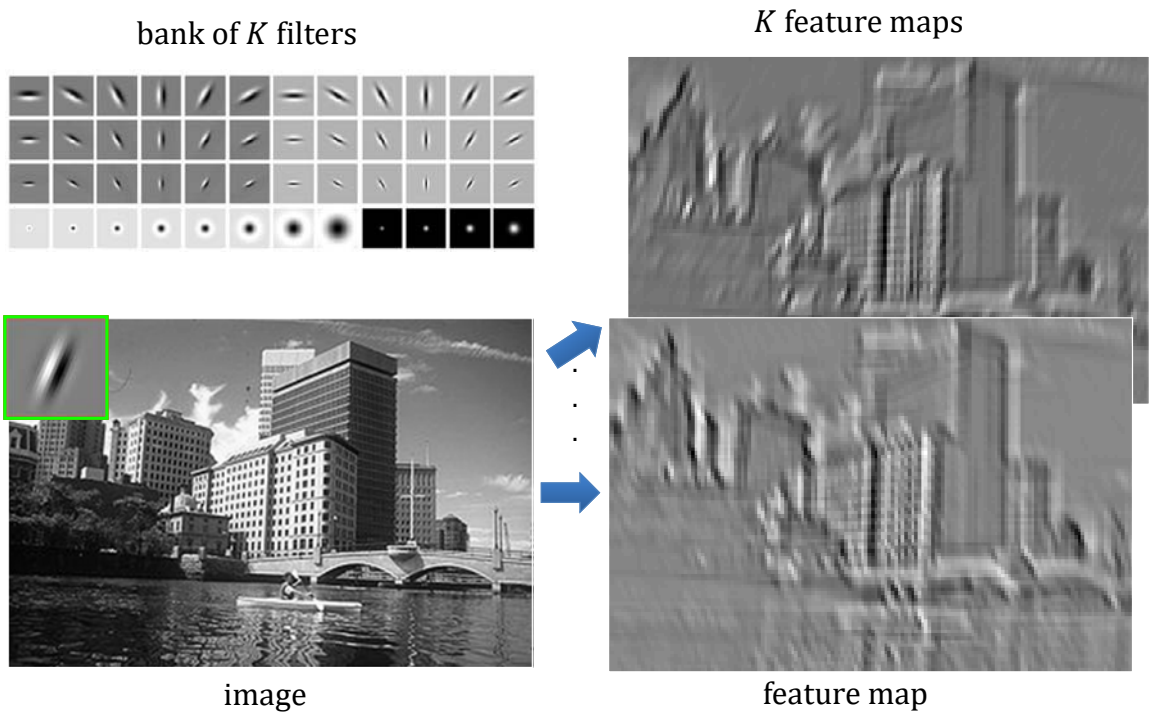
Convolutional layer anatomy



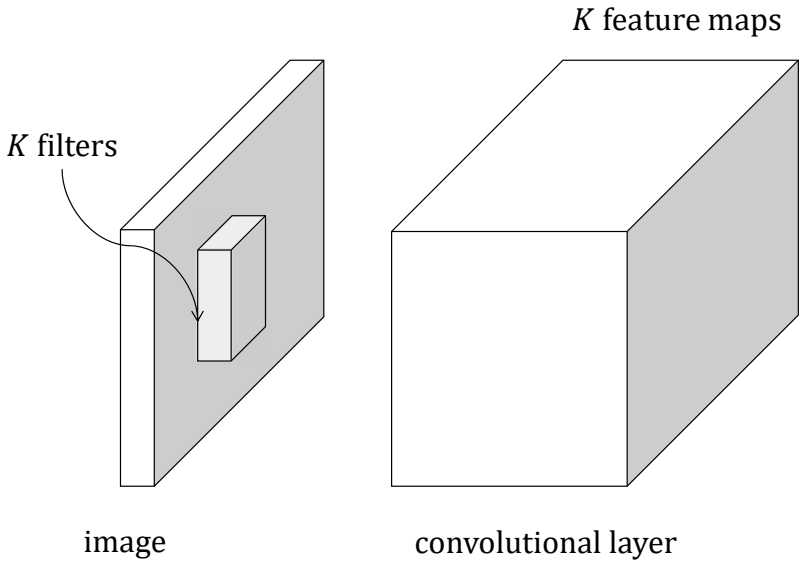
Convolutional layer anatomy



Convolution and traditional feature extraction

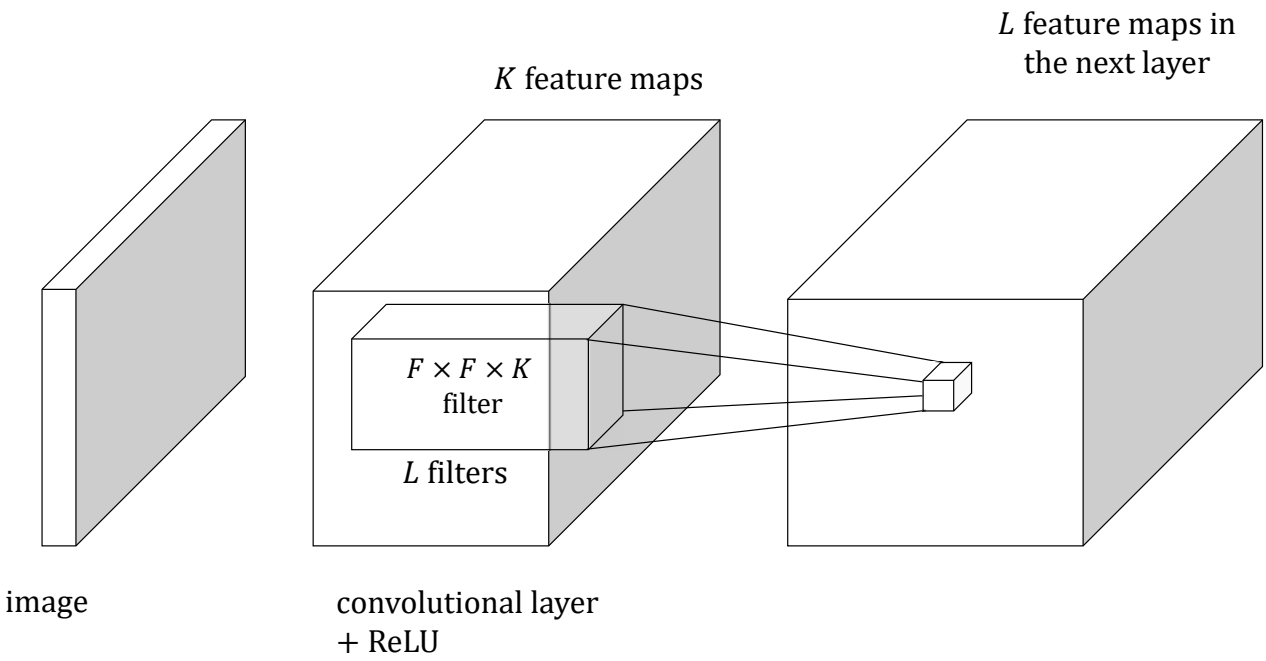


Convolutional layer anatomy

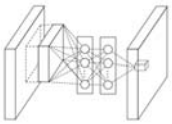


Spatial resolution: (roughly) the same if stride of 1 is used, reduced by $1/S$ if stride of S is used

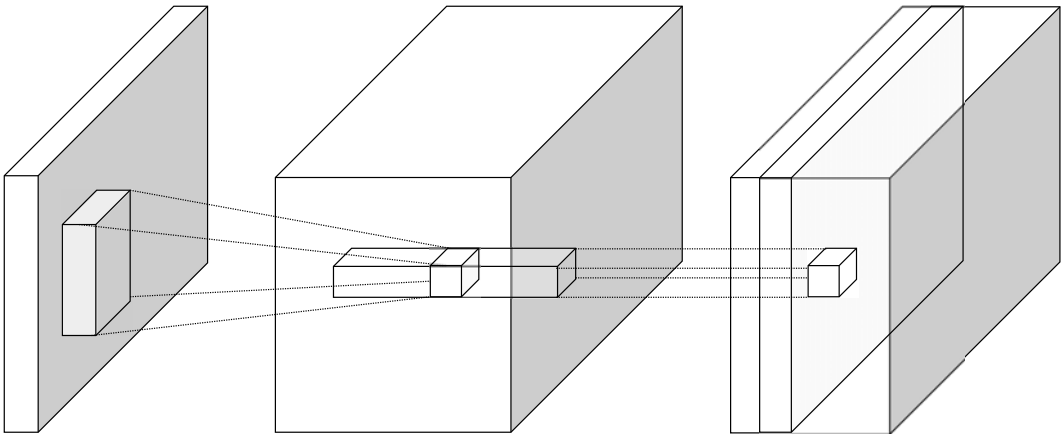
Convolutional layer anatomy



1x1 convolutions

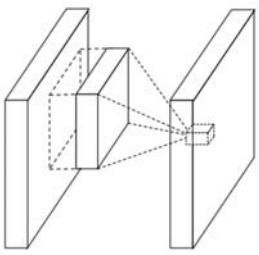


"network in network"

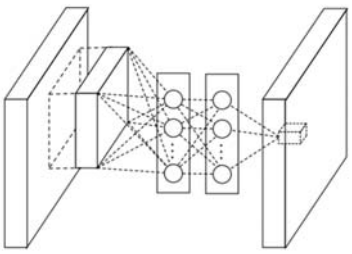


M. Lin, Q. Chen, and S. Yan, Network in network, ICLR 2014

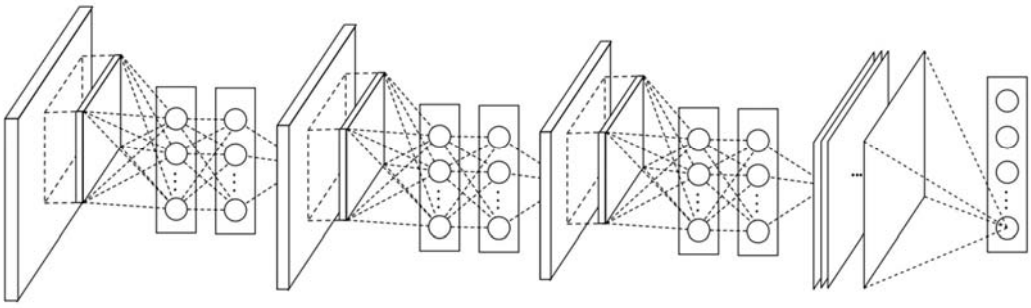
Network in network



(a) Linear convolution layer

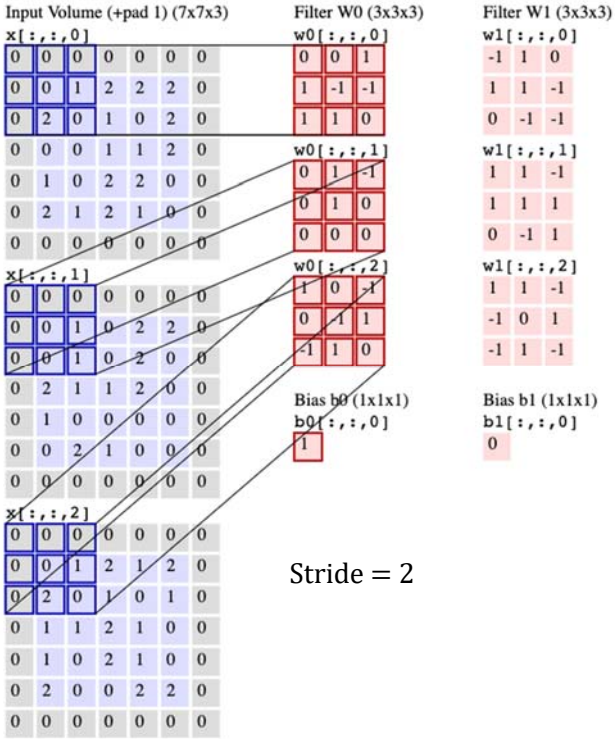


(b) Mlpconv layer

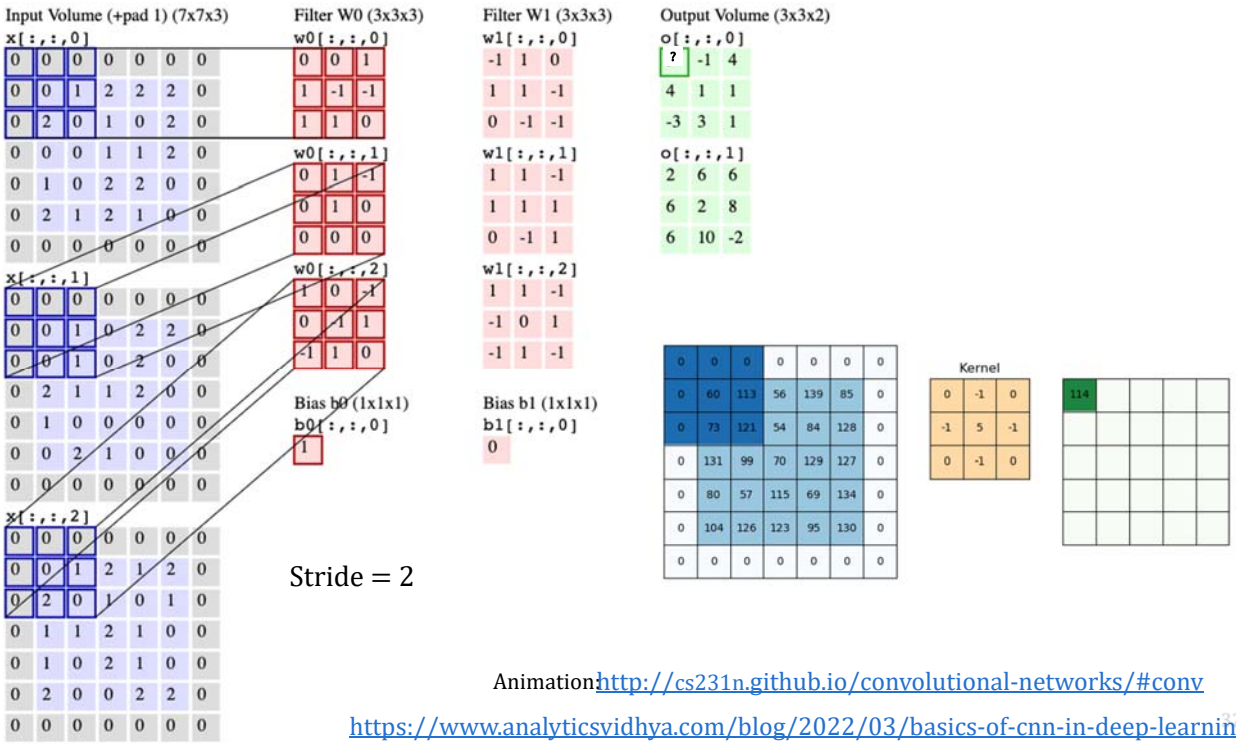


M. Lin, Q. Chen, and S. Yan, Network in network, ICLR 2014

Convolutional layer example



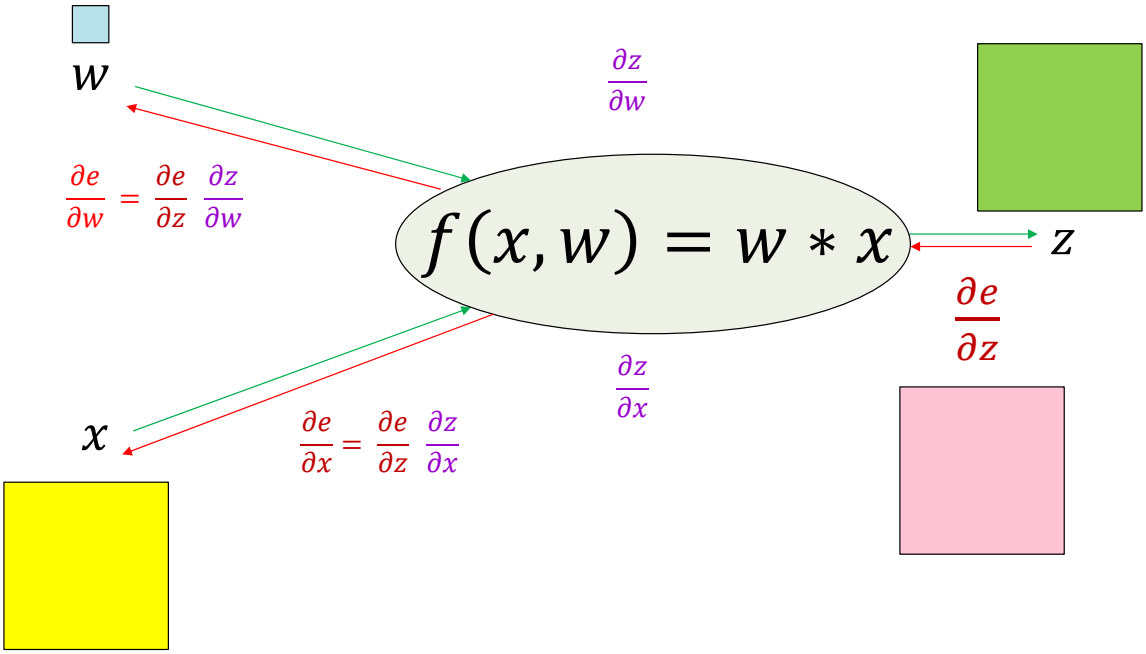
Convolutional layer example



Animation: http://cs231n.github.io/convolutional-networks/#conv

https://www.analyticsvidhya.com/blog/2022/03/basics-of-cnn-in-deep-learning/

Backpropagation for convolutional layer



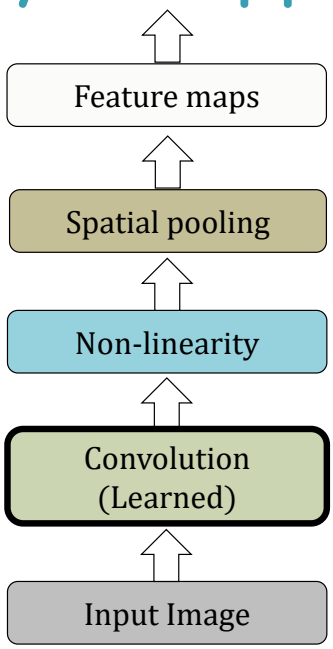
Size of Feature Map

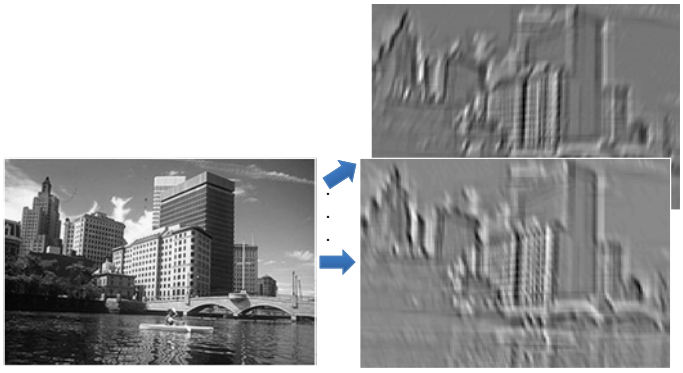
- Output width=((W-Fw+2*P)/S)+1
- Output height=((H-Fh+2*P)/S)+1
- Example
  - Tensor size or shape: (width = 28, height = 28)
  - Convolution filter size (F): (F_width = 5, F_height = 5)
  - Padding (P): 0
  - Stride (S): 1

Feature Map = Input Image x Feature Detector

- Output width=((28-5+2*0)/1)+1=24
- Output height=((28-5+2*0)/1)+1=24
- The output dimension will be (24,24)

Summary: CNN pipeline

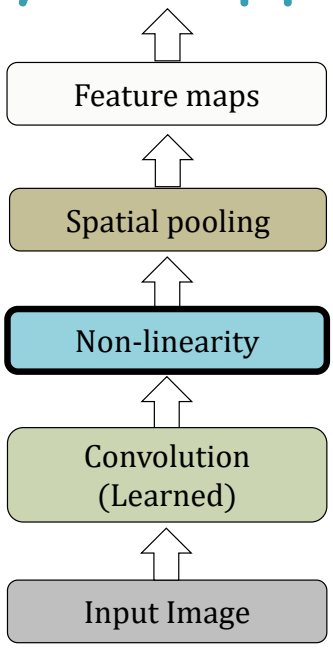




Input Feature Map

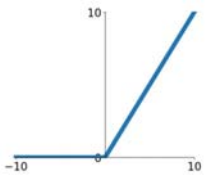
Source: R. Fergus, Y. LeCun

Summary: CNN pipeline



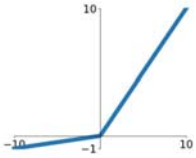
ReLU

$$\max(0, x)$$



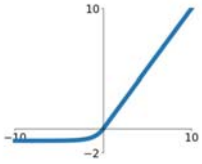
Leaky ReLU

$$\max(0.1x, x)$$



ELU

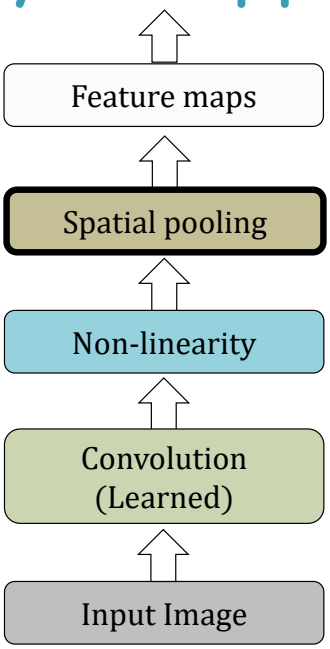
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

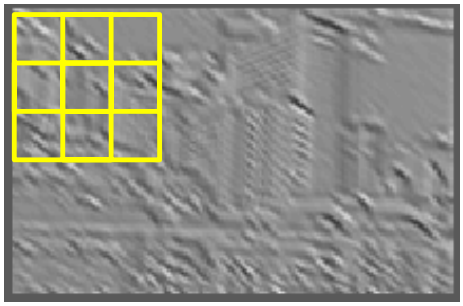


Source: R. Fergus, Y. LeCun

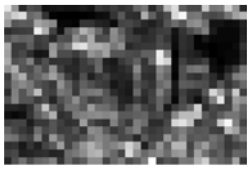
Source: Stanford 231n

Summary: CNN pipeline





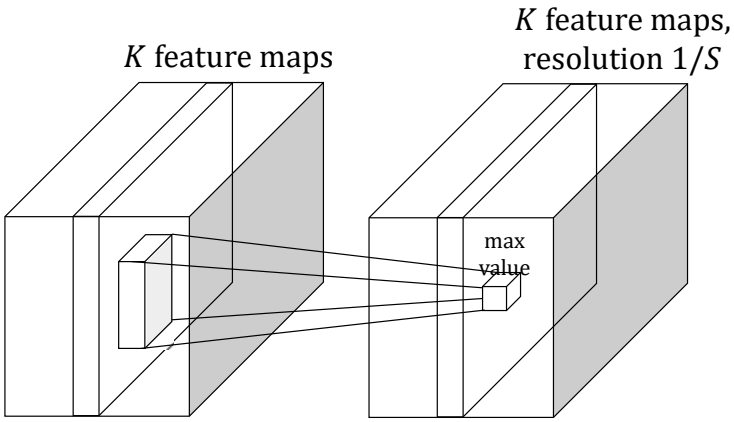
Max
(or Avg)



Source: R. Fergus, Y. LeCun

Pooling

- Spatial Pooling reduces the dimensionality of each feature map but retains the most important information, therefore, controlling overfitting.
- Spatial Pooling can be of different types: Max, Average, Sum etc.
- In practice, Max Pooling has been shown to work better.

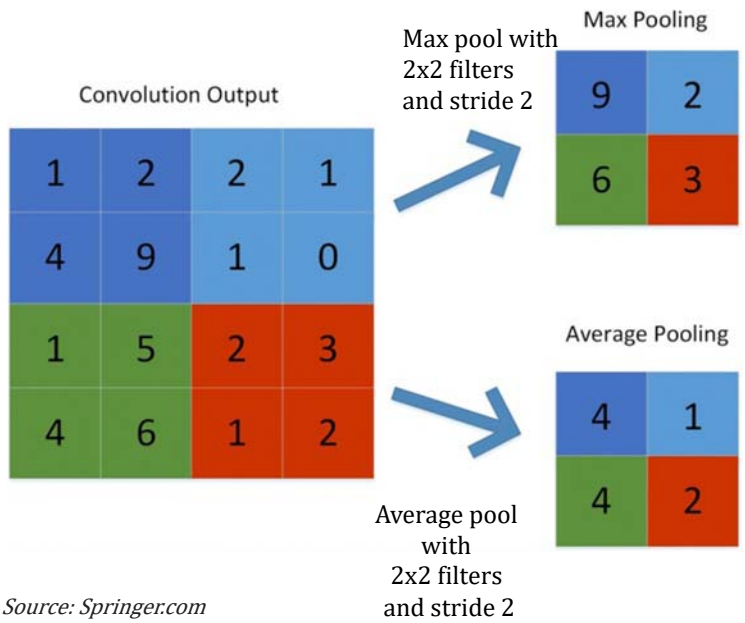


$F \times F$ pooling window, stride S

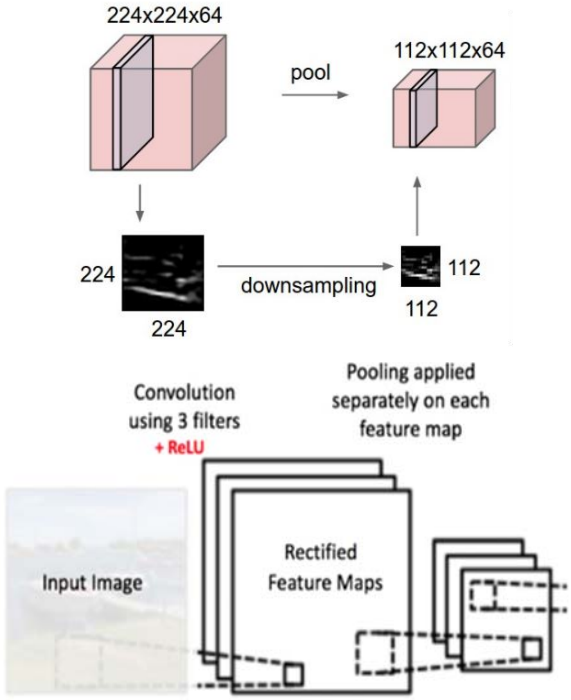
Usually: $F = 2$ or $3, S = 2$

Backward pass: upstream gradient is passed back only to the unit with max value

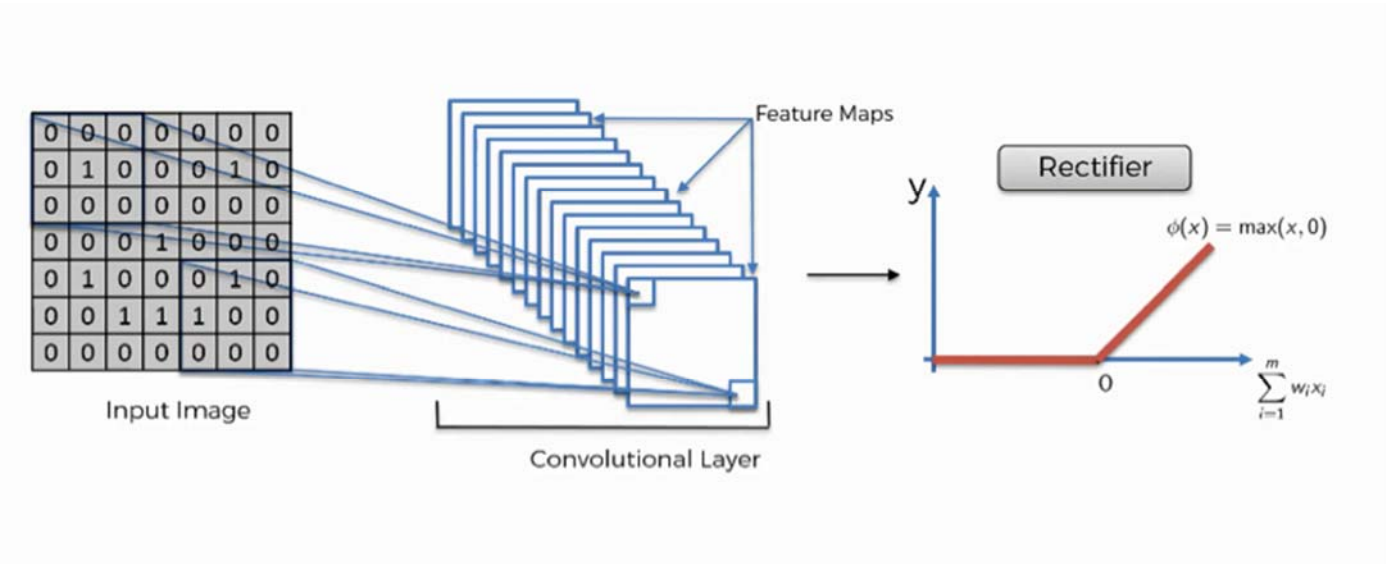
Pooling



Source: Springer.com



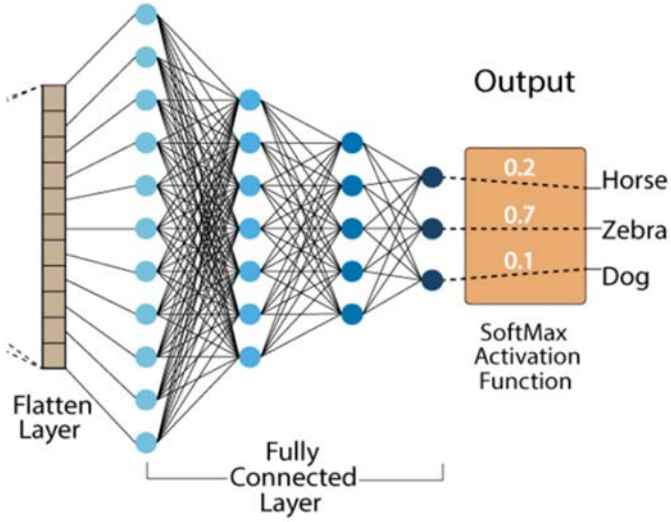
ReLU (Rectified Linear Activation Function)



Source: Superdatascience.com

Fully Connected Layer

- The Fully Connected layer is a traditional Multi Layer Perceptron that uses a softmax activation function in the output layer.
- The term "Fully Connected" implies that every neuron in the previous layer is connected to every neuron on the next layer.
- Softmax classifiers give you probabilities for each class label.



Training the network

- The training process of the Convolution Network may be summarized below:
  - **Step1:** Initialize all filters and parameters/weights with random values
  - **Step2:** Training image as input through the forward propagation step and finds the output probabilities for each class.
  - **Step3:** Calculate the total error at the output layer
  - **Step4:** Use Backpropagation to calculate the *gradients* of the error and use *gradient descent* to update all filter values/weights and parameter values to minimize the output error.
  - **Step5:** Repeat steps 2-4 with all images in the training set.

Testing the network

- When an unseen image is an input into the ConvNet, the network would go through the forward propagation step and outputs a probability for each class (the probabilities are calculated using the weight model from the training).
- If the training set is large enough, the network will generalize well to correctly classify.

Some Network Architectures from ImageNet

- Architectures:
  - LeNet-5 (1990s)
  - 1st generation (2012-2013): AlexNet by Alex Krizhevsky improved from LeNet
  - 2nd generation (2014): VGGNet, GoogLeNet
  - 3rd generation (2015): ResNet
  - 4th generation (2016): ResNeXt, DenseNet
  - 5th generation (2017): SENet
  - Link of new generation, please check
    - *https://keras.io/api/applications/*

Some Network Architectures

- LeNet (1990s)
- 1990s to 2012: In the years from late 1990s to early 2010s convolutional neural network were in incubation. As more and more data and computing power became available, tasks that convolutional neural networks could tackle became more and more interesting.
- AlexNet (2012) – In 2012, Alex Krizhevsky (and others) released AlexNet which was a deeper and much wider version of the LeNet and won by a large margin the difficult ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012. It was a significant breakthrough with respect to the previous approaches and the current widespread application of CNNs can be attributed to this work.
- GoogLeNet (2014) – The ILSVRC 2014 winner was a Convolutional Network from Szegedy et al.from Google. Its main contribution was the development of an Inception Module that dramatically reduced the number of parameters in the network (4M, compared to AlexNet with 60M).

Some Network Architectures

- VGGNet (2014) – The runner-up in ILSVRC 2014 was the network that became known as the VGGNet. Its main contribution was in showing that the depth of the network (number of layers) is a critical component for good performance.
- ResNets (2015) – Residual Network developed by Kaiming He (and others) was the winner of ILSVRC 2015. ResNets are currently by far state of the art Convolutional Neural Network models and are the default choice for using ConvNets in practice (as of May 2016).
- DenseNet (August 2016) – Recently published by Gao Huang (and others), the Densely Connected Convolutional Network has each layer directly connected to every other layer in a feed-forward fashion. The DenseNet has been shown to obtain significant improvements over previous state-of-the-art architectures on five highly competitive object recognition benchmark tasks
- SeNet (2017)
- MobileNet

Some Network Architectures - Parameters

Network	Number of Parameters
LeNet-5	60000
AlexNet	60 million
VGG-16	138 million
GoogleNet	5 million (V1) - 23 million (V3)
ResNet50	25 million
DenseNet-190	40 million
KarNet	16 million

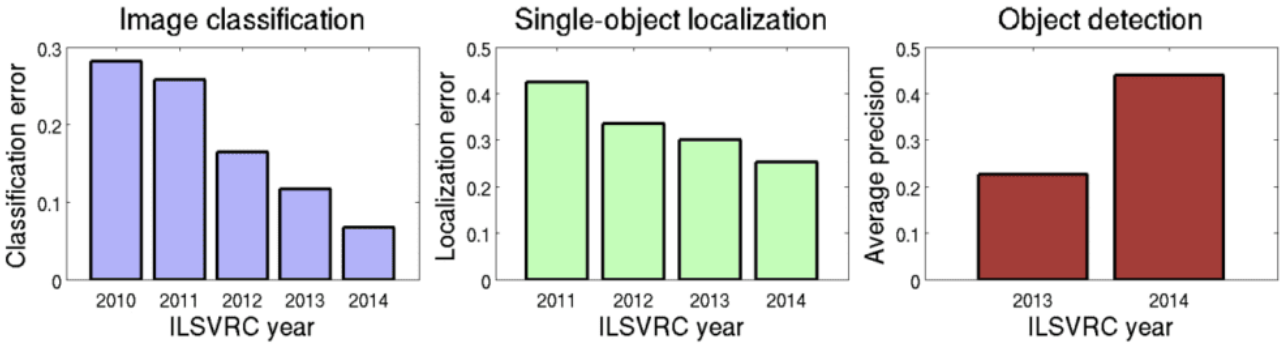
ImageNet Challenge

- 15M images, 22K classes
- Images collected from Web
- Human labelers (Amazon's Mechanical Turk crowd-sourcin
- ImageNet Large Scale Visual Recognition Challenge (ILSVRC-2010)
  - 1K categories
  - 1.2M training images (~1000 per category)
  - 50,000 validation images
  - 150,000 testing images
- RGB images
- Variable-resolution, but this architecture scales them to 256x256 size

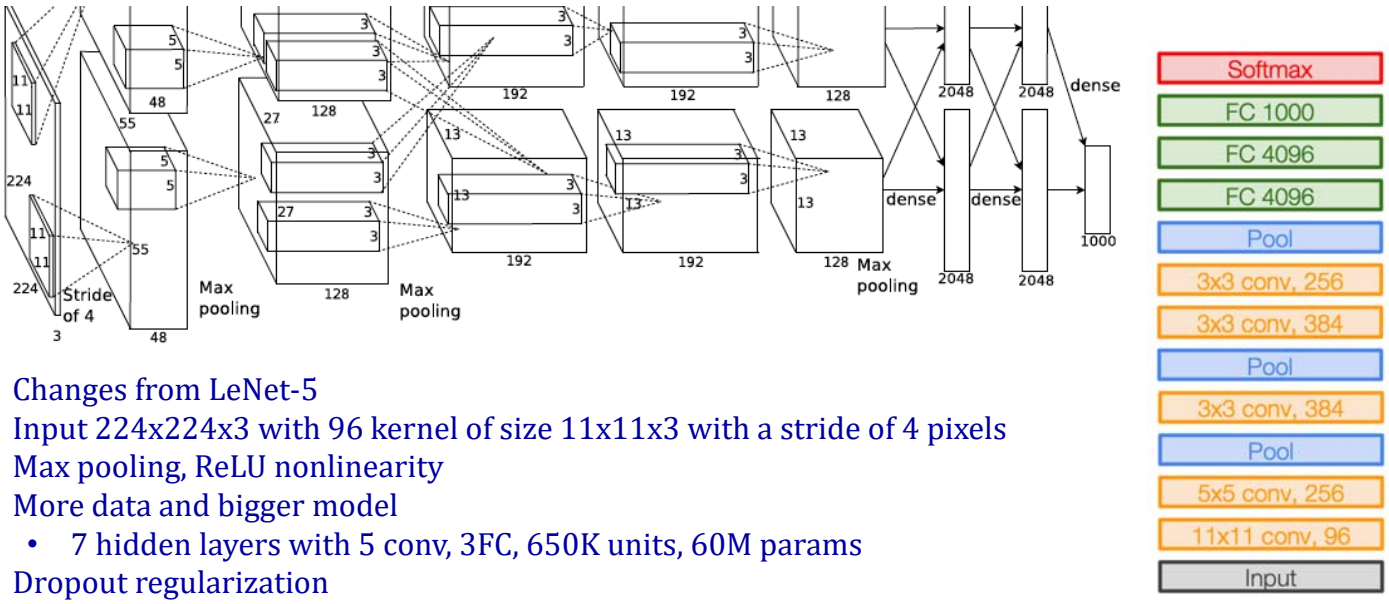
www.image-net.org/challenges/LSVRC/

ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

- General challenge:
  - Image classification
  - Single-object localization
  - Object detection
- **Goals**:
  - Make 1 guess about the label (Top-1 error)
  - Make 5 guesses about the label (Top-5 error)



AlexNet: ILSVRC 2012 winner



- Changes from LeNet-5
- Input 224x224x3 with 96 kernel of size 11x11x3 with a stride of 4 pixels
- Max pooling, ReLU nonlinearity
- More data and bigger model
  - 7 hidden layers with 5 conv, 3FC, 650K units, 60M params
- Dropout regularization
- Trained on two GTX580 3GB GPUs for a week (50x speedup over CPU)

A. Krizhevsky, I. Sutskever, and G. Hinton, ImageNet Classification with Deep Convolutional Neural Networks, NIPS 2012

VGGNet: ILSVRC 2014 2nd place

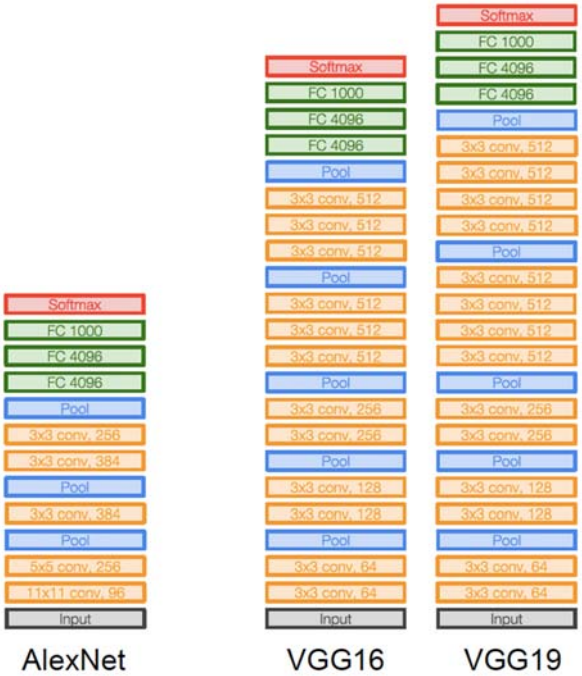


Image source

K. Simonyan and A. Zisserman, Very Deep Convolutional Networks for Large-Scale Image Recognition, ICLR 2015

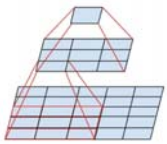
VGGNet: ILSVRC 2014 2nd place

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: **Number of parameters** (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

- Sequence of deeper networks trained progressively
- Large receptive fields replaced by successive layers of 3x3 convolutions (with ReLU in between)



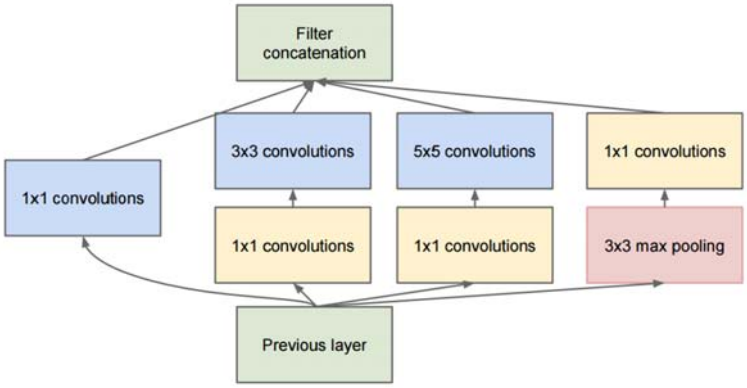
- One 7x7 conv layer with K feature maps needs $49K^2$ weights, three 3x3 conv layers need only $27K^2$ weights
- Experimented with 1x1 convolutions

K. Simonyan and A. Zisserman, Very Deep Convolutional Networks for Large-Scale Image Recognition, ICLR 2015

GoogLeNet: ILSVRC 2014 winner

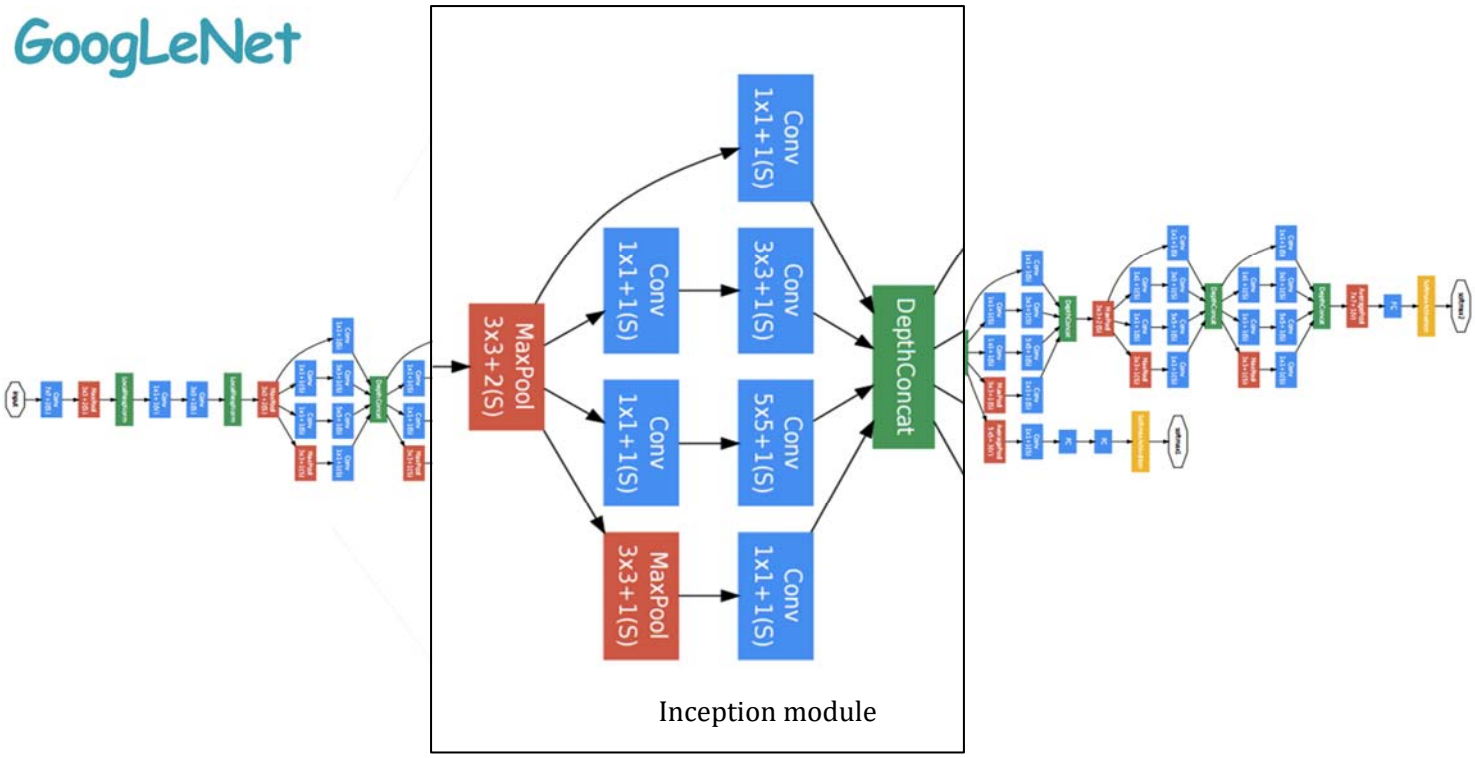
- The Inception Module
  - Parallel paths with different receptive field sizes and operations are meant to capture sparse patterns of correlations in the stack of feature maps
  - Use 1x1 convolutions for dimensionality reduction before expensive convolutions

http://knowyourmeme.com/memes/we-need-to-go-deeper



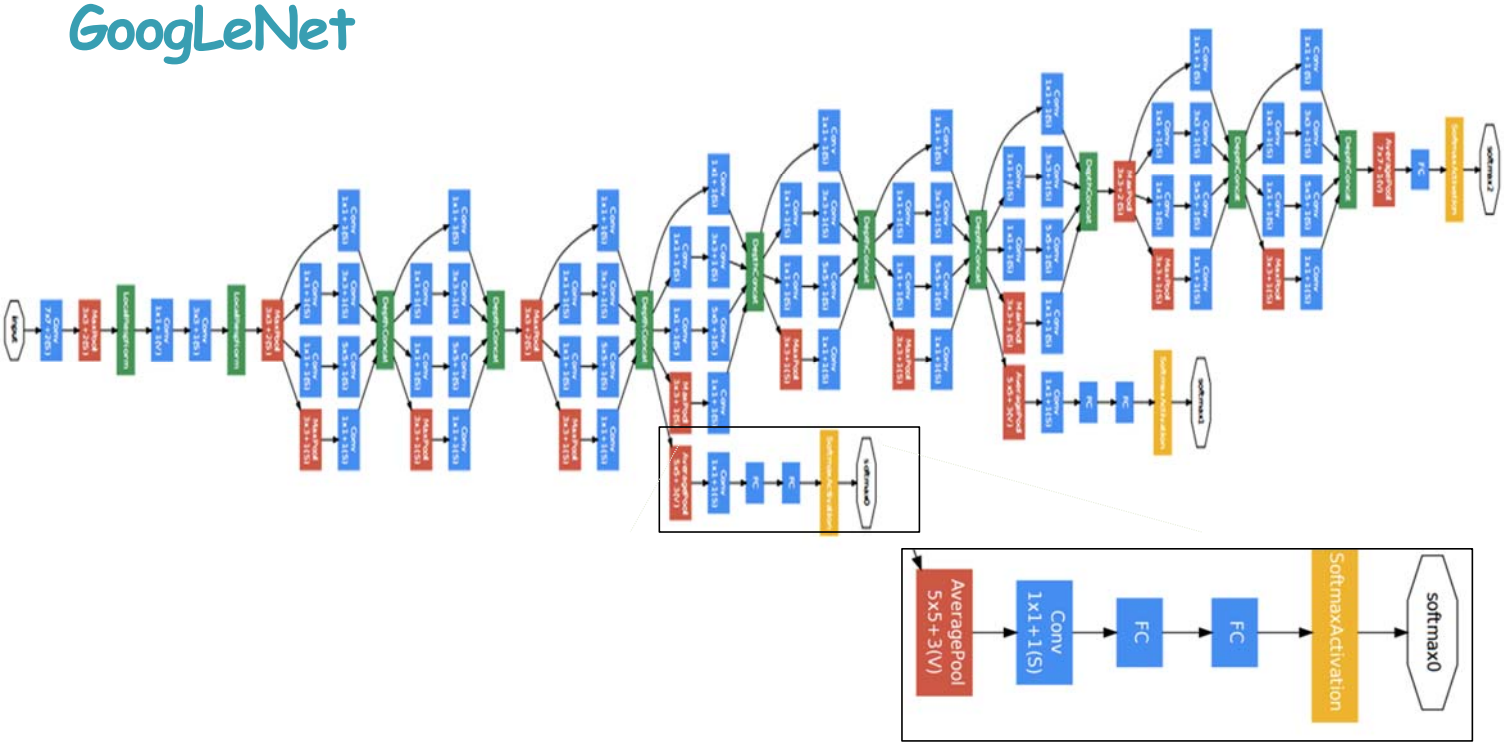
Source: C. Szegedy et al., Going deeper with convolutions, CVPR 2015

GoogLeNet



C. Szegedy et al., Going deeper with convolutions, CVPR 2015

GoogLeNet



Source: C. Szegedy et al., Going deeper with convolutions, CVPR 2015

Auxiliary classifier

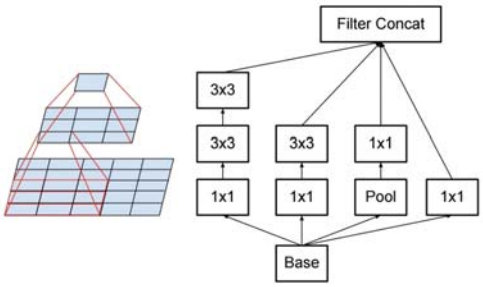
GoogLeNet (Alternative view)

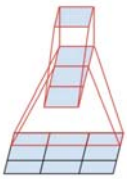
type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

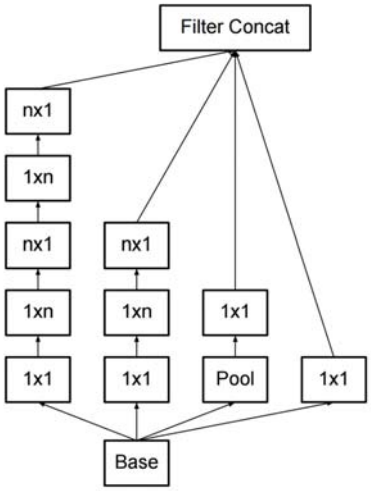
Source: C. Szegedy et al., Going deeper with convolutions, CVPR 2015

Inception v2, v3

- Improve training with batch normalization, reducing importance of auxiliary classifiers
- More variants of inception modules with aggressive factorization of filters
- Increase the number of feature maps while decreasing spatial resolution (pooling)



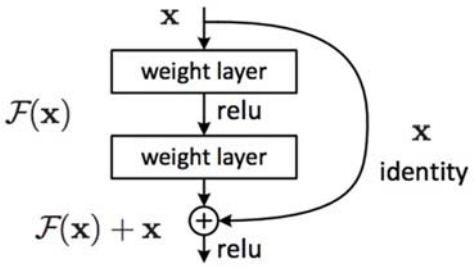


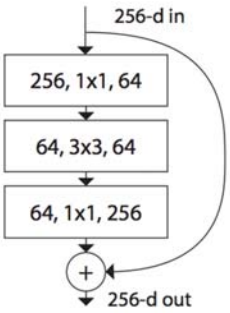


C. Szegedy et al., Rethinking the inception architecture for computer

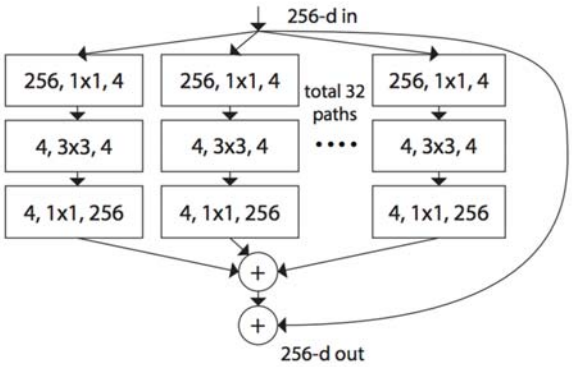
ResNet: ILSVRC 2015 winner

- The residual module
  - Introduce *skip* or *shortcut* connections (existing before in various forms in literature)
  - Make it easy for network layers to represent the identity mapping





ResNet



ResNeXt, roughly same complexity

K. He, X. Zhang, S. Ren, and J. Sun, Deep Residual Learning for Image Recognition, CVPR 2016 (Best Paper)

ResNet

- Architectures for ImageNet:

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

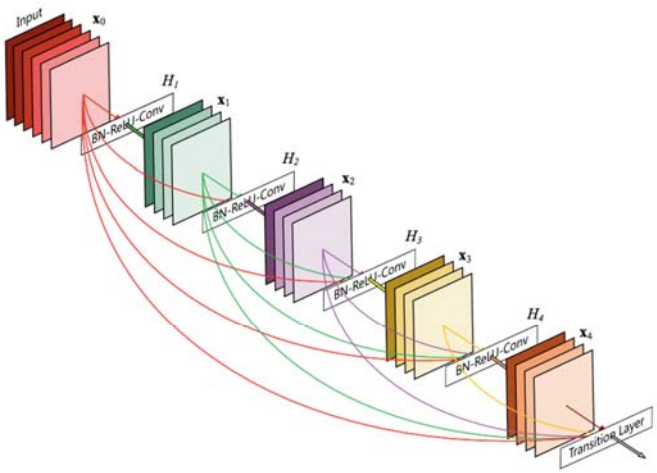
K. He, X. Zhang, S. Ren, and J. Sun, Deep Residual Learning for Image Recognition, CVPR 2016 (Best Paper)

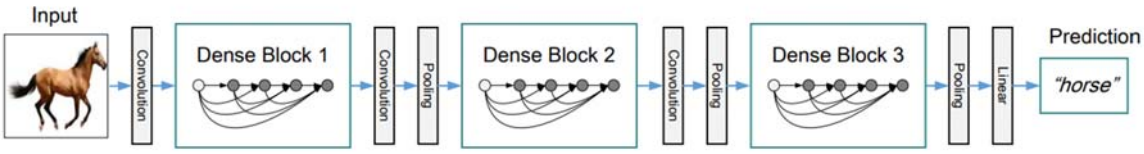
Techniques necessary for state-of-the-art performance

- Training tricks and details: initialization, regularization, normalization
- Training data augmentation
- Averaging classifier outputs over multiple crops/flips
- Ensembles of networks

- (to be covered next time)

Beyond ResNet: DenseNets





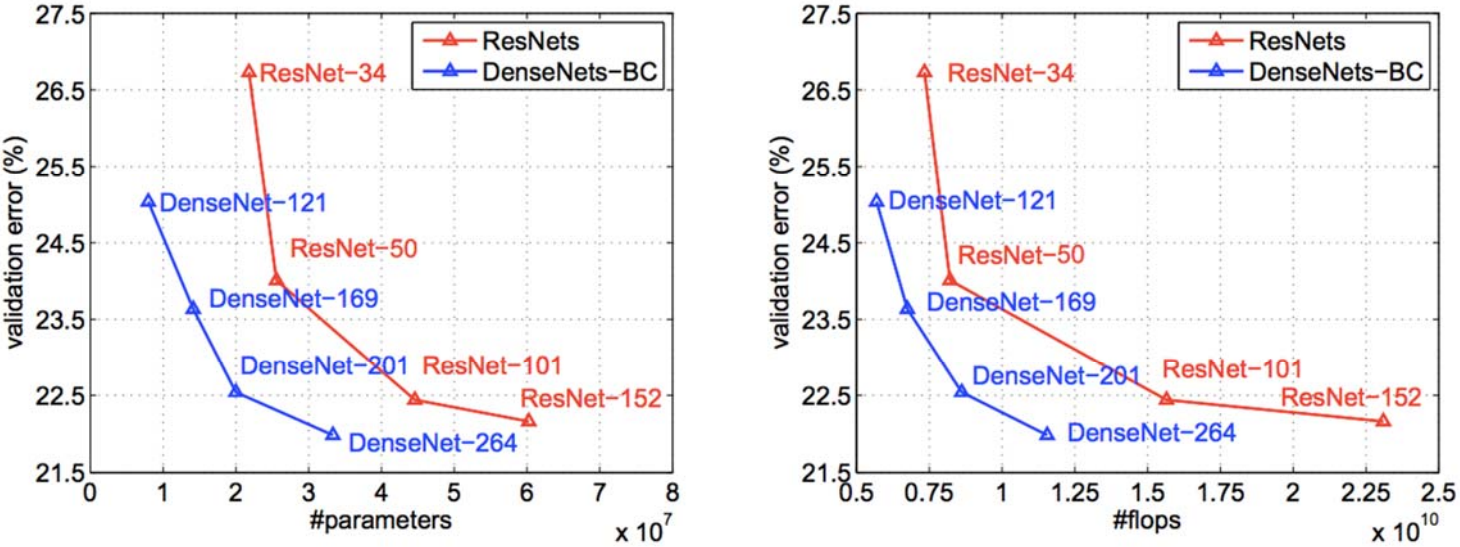
G. Huang, Z. Liu, and L. van der Maaten, Densely Connected Convolutional Networks, CVPR 2017 (Best Paper Award)

DenseNets

Layers	Output Size	DenseNet-121	DenseNet-169	DenseNet-201	DenseNet-264
Convolution	112×112	7×7 conv, stride 2			
Pooling	56×56	3×3 max pool, stride 2			
Dense Block (1)	56×56	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 6$
Transition Layer (1)	56×56	1×1 conv			
	28×28	2×2 average pool, stride 2			
Dense Block (2)	28×28	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 12$
Transition Layer (2)	28×28	1×1 conv			
	14×14	2×2 average pool, stride 2			
Dense Block (3)	14×14	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 24$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 48$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 64$
Transition Layer (3)	14×14	1×1 conv			
	7×7	2×2 average pool, stride 2			
Dense Block (4)	7×7	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 16$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 32$	$\begin{bmatrix} 1 \times 1 \text{ conv} \\ 3 \times 3 \text{ conv} \end{bmatrix} \times 48$
Classification Layer	1×1	7×7 global average pool			
		1000D fully-connected, softmax			

G. Huang, Z. Liu, and L. van der Maaten, Densely Connected Convolutional Networks, CVPR 2017 (Best Paper Award)

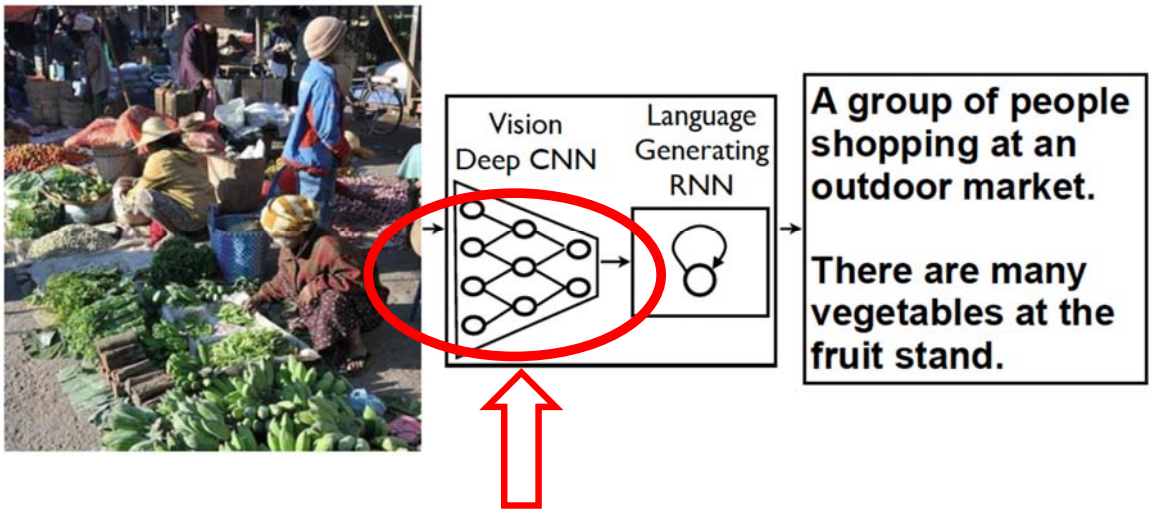
DenseNets



ImageNet validation error vs. number of parameters and test-time operations

G. Huang, Z. Liu, and L. van der Maaten, Densely Connected Convolutional Networks, CVPR 2017 (Best Paper Award)

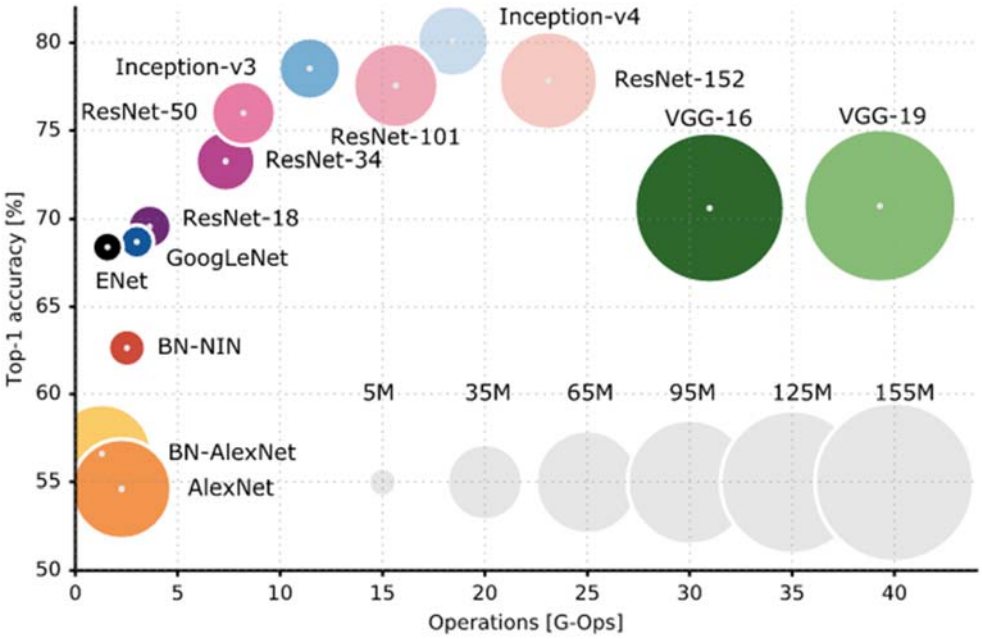
Example: CNNs for image captioning



FC vectors from
pre-trained network

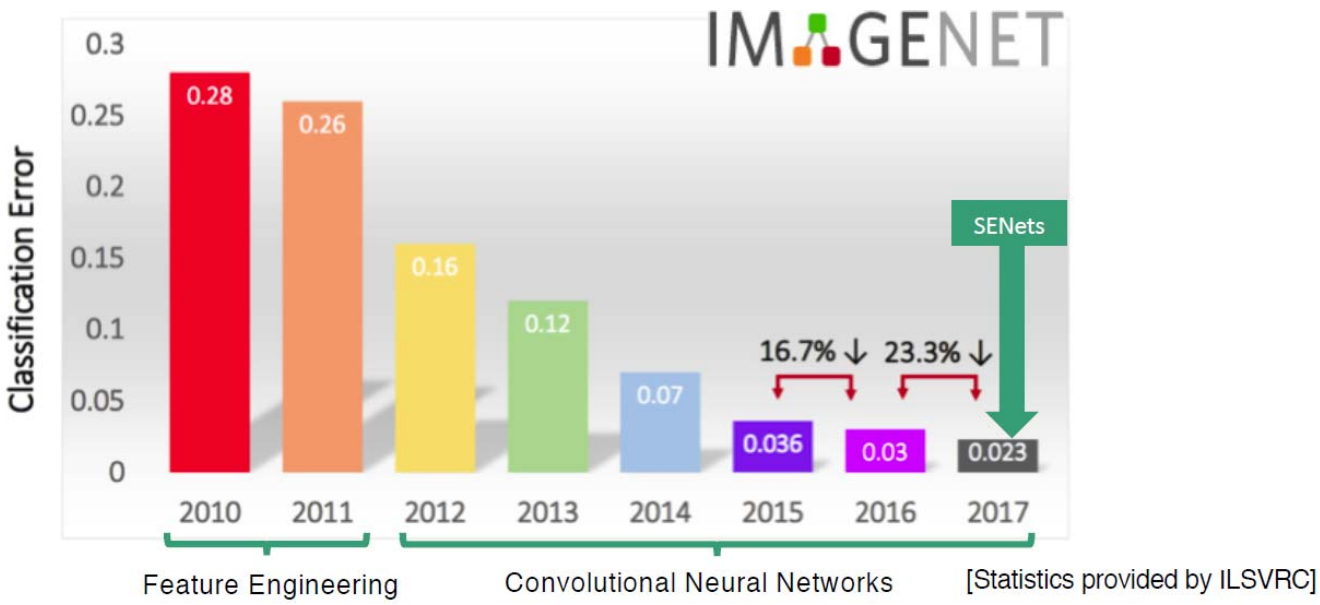
O. Vinyals, A. Toshev, S. Bengio, and D. Erhan. Show and tell: A neural image caption generator. CVPR 2015

Comparing Architectures



https://culurciello.github.io/tech/2016/06/04/nets.html

SeNets



ImageNet Winning CNN Architectures (ILSVRC) | Data Science and Machine Learning | Kaggle

Post-2019

- By 2019, ImageNet classification was nearly solved (Top-5 error ~2%), comparable to or better than **human-level performance** on the task.
- Great catch! Here's an **updated timeline of major ImageNet architectures and improvements post-2019**, highlighting the blend of CNN enhancements and the rise of Transformers:
- **CNN & Hybrid Architectures (Post-2019)**
  - **EfficientNet-V2** (2021) – Improved version of EfficientNet with further architectural refinements from NAS and enhanced training techniques like progressive resizing, RandAugment, and Mixup.
  - **FixEfficientNet** (2020) – Enhances performance of EfficientNet via improved training regimes (FixRes), achieving up to **88.5% Top-1 accuracy**, and **98.7% Top-5 accuracy** on ImageNet with a single crop.
  - **Big Transfer (BiT)** (2020) – A scalable variant of ResNet (e.g., ResNet-152×4) pretrained on massive datasets like JFT-300M, using group normalization and weight standardization for better transfer learning.

Post-2019

- **Vision Transformers (ViTs) and Hybrid Models**
  - **DeepViT** (2021) – Addresses "attention collapse" in deep ViT models using Re-attention, improving training stability and boosting **Top-1 accuracy by ~1.6%** on ImageNet.
  - **CSWin Transformer** (2021) – A hierarchical Transformer backbone using cross-shaped window attention. Achieves **85.4% Top-1 accuracy** on ImageNet-1K (no extra data), with further improvement to **87.5%** when pretrained on ImageNet-21K.
  - **LeViT** (2021) – Hybrid model combining CNN-like features with Transformers for fast inference. At **~80% Top-1 accuracy**, LeViT is reportedly **5× faster** than EfficientNet on CPU hardware.

Installation

Packages	Installation	Link documents
Python 3.9.x up	Installer 64 bits	https://www.python.org/
numpy	pip install --upgrade numpy	http://www.numpy.org/
sciPy	pip install --upgrade scipy	https://www.scipy.org/
Matplotlib	pip install --upgrade matplotlib	https://matplotlib.org/
Pandas	pip install --upgrade pandas	http://pandas.pydata.org/
scikit-learn	pip install --upgrade scikit-learn	http://scikit-learn.org/stable
TensorFlow 2.X	pip install --upgrade tensorflow	https://www.tensorflow.org
Keras	pip install --upgrade keras	https://keras.io/

Wheel link: http://www.lfd.uci.edu/~gohlke/pythonlibs/
***Set path of python in environment variable,
C:\Users\...\AppData\Local\Programs\Python\Python3X\Scripts\
C:\Users\...\AppData\Local\Programs\Python\Python3X\

Installation

Packages	Installation	Link documents
Jupyter	pip install --upgrade jupyterlab	http://jupyter.org
OpenCV	pip install --upgrade opencv-python	https://pypi.org/project/opencv-python/ https://pythonexamples.org/python-opencv/
Pytorch		https://pytorch.org/

GPU Tools	Link documents
NVIDIA® GPU drivers	CUDA® architectures 3.5, 5.0, 6.0, 7.0, 7.5, 8.0 and higher than 8.0 https://www.nvidia.com/download/index.aspx?lang=en-us
CUDA® Toolkit (64bits)	TensorFlow supports CUDA® 11.2 (TensorFlow >= 2.5.0) https://developer.nvidia.com/cuda-toolkit-archive
cuDNN SDK 8.1.0	https://developer.nvidia.com/rdp/cudnn-archive

References

- http://introtodeeplearning.com/
- https://www.deeplearningbook.org/
- https://cs231n.github.io/
- http://slazebni.cs.illinois.edu/fall20/
- Tutorials | TensorFlow Core
- Keras Applications
- (Tutorial) KERAS Tutorial: DEEP LEARNING in PYTHON – DataCamp
- Illustrated: 10 CNN Architectures | by Raimi Karim | Towards Data Science